FICHE MÉMO

Mémo du module 2

Ce que vous retenez des deux jours : un harness outillé et borné, une context window maîtrisée, la spec (et la doc) comme patrimoine, et des pratiques partagées en équipe.

1. Boîte à outils d'agents personnels

Agent de code = Modèle + Harness. Le modèle raisonne ; le harness lui donne les moyens d'agir : boucle d'outils, contexte et mémoire, permissions et sandboxing, sous-agents et hooks, observabilité. Un même modèle dans deux harness différents peut produire des résultats très différents.

Brique	Rôle	Activation
`AGENTS.md` (`CLAUDE.md` , `copilot-instructions.md`)	Règles stables, conventions, structure du projet	toujours actif
Agent Skills (`SKILL.md`)	Savoir-faire réutilisable, avec scripts embarqués	à la demande
Hooks	Scripts déterministes sur les événements du cycle de vie	événementiel
Agents custom / sous-agents	Périmètre borné, contexte isolé	isolation
MCP	Brancher l'agent sur l'écosystème (forge, doc, ticketing)	outils

Un concept, plusieurs harness :

Concept	GitHub Copilot	Claude Code
Instructions de projet	`.github/copilot-instructions.md` · `AGENTS.md`	`CLAUDE.md` · `AGENTS.md`
Skills	`.github/skills/<nom>/`	`.claude/skills/<nom>/`
Sous-agents	`.github/agents/*.agent.md`	`.claude/agents/` + outil `Task`
Hooks	`.github/hooks/*.json`	`.claude/settings.json`
MCP	`mcp-config.json`	`.mcp.json`
Context window	`/context` · jauge VS Code	`/context` · `/usage`
Mode non interactif	`copilot -p "…"`	`claude -p "…"`

Piloter l'autonomie : observer → demander à chaque action → laisser faire dans le périmètre → ne plus rien demander → aller au bout seul. Le mode n'est qu'un défaut : les droits vivent dans un fichier (`--allow-`

`tool`, `permissions.allow/ask/deny`, `config.toml`). Une allowlist n'est pas une sandbox : le seul périmètre réel est l'isolation.

> La policy vit dans le runtime, pas dans le prompt : un hook refuse ce qu'une consigne d'`AGENTS.md` ne fait que suggérer.

2. Mémoire, tokens et indexation

- Le **KV Cache** évite de recalculer l'historique ; il vit côté serveur et expire (5 min chez Anthropic, 24 h ou 30 min chez OpenAI selon le modèle). Un cache expiré coûte de l'argent et de la latence, jamais de la mémoire.
- **Mesurer d'abord** : `/context` (Copilot, Claude Code), `/status` (Codex). La dépense se lit ailleurs : AI Credits (Copilot), `/cost` (Claude Code), `/usage` (Codex), `ccusage`.
- **Réduire** : `/compact [focus on …]`, `/clear` quand la tâche est finie, prompt caching, agents custom, skills à la demande, modèle choisi au démarrage (`/model` en cours de session invalide le cache).
- **Lire les chiffres avec esprit critique** : rtk (gain annoncé –60 à –90 %, mesuré +7,6 % de coût) ; context-mode tient (–97,5 %) ; ponytail (–10,3 % de coût) rapporte plus que caveman (–8,5 %).
- **Mémoire** : fichier versionné (`AGENTS.md`), mémoire native (Auto Memory, Copilot Memory, Codex), middleware (`memoir`). `/memory` l'ouvre dans Claude Code comme dans Copilot CLI. Aucun standard : changer d'agent coûte la mémoire.
- **RAG sur le code** : chunking naïf, AST-aware, knowledge graph. Graphify : `graphify install --platform copilot`, `graphify .`, `graphify hook install`.

3. Spec-Driven Development : fondations

- Les assistants **formalisent** les user stories, ils ne remplacent pas les conversations avec les utilisateurs.
- Sans guidelines, une story générée est verbeuse, hors syntaxe, trop technique, sans Definition of Done : écrivez votre guide (skill `story-writing`).
- Trois frameworks : **OpenSpec** (léger, `propose → apply → archive`), **Spec Kit** (`/speckit.constitution → specify → plan → tasks → implement`, le plus adaptable), **BMAD** (20+ agents, Quick Flow ou Standard Flow).
- Règle minimale : la **spécification précède la génération de code**.

4. Workflow adapté à l'IA

- Trois postures : **spec-first → spec-anchored → spec-as-source**. Aucun framework n'atteint encore le spec-as-source.
- Customiser sans forker : Spec Kit (extensions = nouvelles capacités, presets = surcharges, bundles = distribution) ; BMAD (overrides TOML, expansion packs) ; OpenSpec (config projet `openspec/config.yaml`, schemas custom dans `openspec/schemas/`).
- La spec n'est qu'une partie de la doc : la **documentation complète** (fonctionnelle et technique) est le patrimoine, le code un artefact régénérable.
- Trois garde-fous : **tests**, **revue agentique** par un agent distinct, **le commit** (doc à jour, part d'IA tracée, `Co-Authored-By:`).

- Tests de construction (pour l'agent, jetables) vs tests fonctionnels (pour l'équipe, durables) ; le score de mutation complète la couverture.

5. Mise en pratique sur un projet

- Ce qui coûtait deux semaines en coûte deux jours : l'outil interne « pas assez rentable » le devient (framework de tests d'API ou E2E).
- Migrer un legacy : re-documenter d'abord, puis refondre par étapes (Strangler Fig, Branch by Abstraction, Anti-Corruption Layer, Parallel Run).
- Sur un projet existant : skills et instructions → constitution → specs sur les nouvelles features → rétro-documentation du seul périmètre touché.

6. Le futur du dev : coder autrement

- Superviser au lieu d'approuver : poste terminal-first (multiplexeur de terminal au choix, `lazygit` pour relire le diff).
- Git worktrees : `git worktree add -b feat/x ../proj-x`, un agent par worktree ; les fichiers non versionnés (`.env` , `node_modules/`) ne suivent pas.
- Au-delà de deux sessions parallèles, on ne supervise plus, on subit.
- Le coût caché : l'érosion de la connaissance. La revue de code évolue, elle ne disparaît pas.

7. Partage & mutualisation en équipe

- Skills versionnés dans le dépôt (`.github/skills/` , `.claude/skills/` , `.agents/skills/`), distribués par `gh skill install <repo> <skill>@<tag>` ou skills.sh.
- Description à la troisième personne, ce que ça fait **et** quand l'utiliser ; test « agent A écrit, agent B teste ».
- Investir si c'est **spécifique**, **répété** et **maintenu** ; sinon attendre.
- `AGENTS.md` d'équipe : socle commun prescriptif, tout écart documenté.

> Le fil rouge : la spec (et la doc) est le patrimoine, le code est un artefact régénérable.

SUPPORT D'EXERCICE

Prérequis des labs

Ce qu'il faut installer et vérifier avant le J1, pour GitHub Copilot comme pour Claude Code. Les concepts des labs sont identiques dans les deux harness : seuls les emplacements de fichiers et les commandes changent (table de correspondance dans les slides, Ch. 1, « un concept, plusieurs harness »).

Le harness

GitHub Copilot

- Une **licence GitHub Copilot** active (Pro, Business ou Enterprise). Elle est nécessaire **y compris pour le CLI**.
- **GitHub Copilot CLI** installé et authentifié. Trois canaux d'installation, le même outil au bout :

```bash
brew install --cask copilot-cli          # macOS / Linux
npm install -g @github/copilot           # tout OS, exige Node.js 22+
curl -fsSL https://gh.io/copilot-install | bash   # script d'install
# Windows : winget install GitHub.Copilot

copilot --version   # vérifier que la commande répond
copilot             # doit ouvrir une session interactive ; /login si non authentifié
```

> Piège de nom sur Homebrew : `brew install copilot` installe **AWS Copilot** (déploiement ECS), qui n'a aucun rapport. Il faut bien `--cask copilot-cli`. Le canal d'installation n'a en revanche aucune incidence sur les labs : `brew` livre le même binaire que `npm`, sans exiger d'environnement Node.

Trois vérifications à faire **avant** la session (ce sont les blocages classiques du J1) :

1. La licence Copilot couvre bien le CLI, et pas seulement l'IDE.
2. Sur les plans Business / Enterprise, **Copilot Memory** et les **agent skills** doivent être activés par un administrateur de l'organisation.
3. L'authentification passe par `/login` dans la session, ou par une variable `GH_TOKEN` / `GITHUB_TOKEN` / `COPILOT_GITHUB_TOKEN`.

Claude Code

- Une **licence Claude Code** active (Claude Pro, Max, Team ou Enterprise), ou un accès à l'**API Anthropic** via une clé de la Console. L'un ou l'autre suffit pour authentifier le CLI.
- **Claude Code CLI** installé et authentifié. Deux canaux d'installation, le même outil au bout :

```bash
npm install -g @anthropic-ai/claude-code          # tout OS, exige Node.js 18+
curl -fsSL https://claude.ai/install.sh | bash    # script d'install natif (macOS/Linux/WSL)

claude --version   # vérifier
claude             # doit ouvrir une session interactive ; /login si non authentifié
```

Trois vérifications à faire **avant** la session :

1. L'authentification passe soit par `/login` (compte claude.ai), soit par une variable `ANTHROPIC_API_KEY` (accès API direct) : choisissez un seul mode et tenez-vous-y pour toute la session, les deux ne se combinent pas.
2. Sur les plans Team / Enterprise, l'accès aux MCP externes et aux hooks peut être restreint par la politique d'administration de l'organisation.
3. Lancez `claude` depuis la racine du projet, sans quoi le contexte de départ est vide.

Socle commun (tous les labs)

Outil	Pourquoi	Vérifier
Git	Tous les labs. Plusieurs supposent un dépôt **initialisé** (`git init` au minimum)	`git --version`
Node.js + `npm` / `npx` (22+ pour Copilot CLI, 18+ pour Claude Code)	CLI (canal npm), serveurs MCP en `npx`, OpenSpec	`node --version`
Python 3.10+ + `uv`	Graphify, Spec Kit, Schemathesis	`python3 --version` · `uv --version`
GitHub CLI (`gh`), version récente (mi-2026 ou postérieure)	`gh skill` et la PR du lab 9	`gh --version`
Un **navigateur**	Ouvrir le graphe Graphify, l'app Pomodoro, l'interface E2E	—

Par lab : ce qu'il faut en plus

Lab	Outillage spécifique	Sans quoi
1.1 Outils MCP	Socle. Un **dépôt git** (sinon les outils GitHub n'ont rien à lire). Claude Code : un **jeton GitHub** (`GITHUB_TOKEN`), lab facultatif	Le serveur MCP répond dans le vide
1.2 Hook local	Un shell `bash` (ou `powershell` sous Windows). Droit d'exécution sur les scripts (`chmod +x`)	Le hook est chargé mais ne fait rien
1.3 Graphify	`uv` ou `pip` → paquet `graphifyy`. Un navigateur pour `graph.html`	Pas de graphe
2.1 / 2.2 User stories	Rien de plus	—
3.1 Pomodoro OpenSpec	`npm install -g @fission-ai/openspec@latest`	—
3.2 Pomodoro Spec Kit	`uv tool install specify-cli` (depuis git → **accès réseau à GitHub**)	—

Lab	Outillage spécifique	Sans quoi
4.1 → 4.6 Extensions Spec Kit	Spec Kit (3.2) + un projet déjà initialisé par `specify init`	`specify extension add` échoue
5.2 Pilotage E2E	**Docker + docker-compose** · **Playwright** (télécharge ses navigateurs au premier lancement, ~400 Mo) · une **clé d'API LLM** (voir ci-dessous)	Le TP ne démarre pas
5.3 Variante contrat d'API	`pip install schemathesis` (ou Pact) · même clé d'API · une spec OpenAPI	Idem
6 Rétro-documentation	`npx reversa` (Node) · le projet legacy choisi et **ses** prérequis (pour `spring-petclinic-angularjs` : **JDK + Maven + Node**)	Le projet ne se lance pas
7 Poste terminal-first *(facultatif)*	`tmux` , `yazi` , `lazygit` . Optionnel pour yazi : `ffmpegthumbnailer fd ripgrep fzf zoxide`	—
8 Développement parallèle	Git worktrees (natif). Une PetClinic qui **démarre**, donc son runtime (JDK + Maven, ou Python)	L'agent débugge un faux problème
9 Skill d'équipe	`gh` authentifié + un dépôt GitHub où pousser une PR	Pas de mise en situation

Clé d'API LLM (labs 5.2 et 5.3)

C'est l'**application que vous construisez** qui appelle un modèle, pas votre agent. Trois options, à préparer **avant** la session :

- **Google AI Studio (Gemini)** : gratuit et le plus généreux en quota, le chemin le plus court aujourd'hui ;
- **OpenRouter** : palier gratuit limité (quelques dizaines de requêtes par jour, vite atteint sur ce TP) ;
- **votre propre clé d'API**, chez le fournisseur de votre choix.

> Sur les modèles gratuits, les données sont fréquemment traitées hors UE : **aucune donnée sensible ou client** dans ce lab. Votre licence Copilot ou votre abonnement Claude Code ne couvre **pas** cet usage : il sert à développer, pas à alimenter le produit livré.

> **Si vous avez déjà une clé d'API, c'est la meilleure option**, et de loin la plus proche de la réalité d'un projet : ni quota gratuit à ménager ni modèle imposé, et si la clé porte sur une offre payante ou un point d'accès en UE, la mise en garde sur les données ne s'applique plus. C'est aussi le seul montage transposable tel quel après la formation. Le framework du lab s'attend à un point d'accès **compatible OpenAI** ; la plupart des fournisseurs en exposent un, sinon l'adapter fait partie du TP.

Runtime de la codebase

Les labs 1.3, 7 et 8 travaillent sur la **PetClinic** (choix de la version : handout « Lab 1 »). Les labs 1.3 et 8 exigent qu'elle **compile et démarre** : prévoyez le runtime correspondant (JDK + Maven pour la version Java, Python pour la version Django). Les autres labs (1.1, 1.2) se contentent d'un dépôt contenant du code.

SUPPORT D'EXERCICE

Lab 1 · Boîte à outils agents personnels

Repartir avec un écosystème agent personnel opérationnel : outils branchés et bornés (1.1), garde-fous déterministes (1.2), codebase indexée (1.3). Chaque étape a sa déclinaison GitHub Copilot et Claude Code.

Avant de commencer

La codebase fil rouge

Les labs de ce chapitre, puis les labs 7 et 8, travaillent sur la **PetClinic**. Trois options :

1. **Votre propre PetClinic**, si vous en avez déjà une : c'est le chemin le plus court.
2. **La PetClinic Java officielle** : https://github.com/spring-projects/spring-petclinic (Spring Boot, Maven). C'est la référence, la mieux maintenue et la plus documentée.
3. **Une PetClinic Python**, si Java n'est pas votre monde : https://github.com/antonarhipov/django-petclinic (Django, SQLite, mêmes entités `owners` / `pets` / `vets` / `visits`).

> Les labs **2.2**, **7** et **8** vous font **modifier** la codebase (nouvelle entité `Clinic`, champ `phone` sur `Owner`, deux features en parallèle). **Forkez** donc le dépôt choisi dans votre espace personnel (en dépôt **privé** si vous préférez) plutôt que de travailler sur un clone que vous ne pourrez pas pousser. Le lab 8 (worktrees) et le lab 9 (PR sur un dépôt partagé) supposent que vous avez les droits d'écriture.

> Sur l'alternative Python : l'écosystème est mince, aucun portage n'a de communauté réelle et **aucun des candidats n'a de licence explicite**, à garder en tête avant de forker dans un contexte client. `antonarhipov/django-petclinic` est retenu parce qu'il est le plus récent, qu'il utilise **SQLite** (aucune base à installer) et que son découpage en applications Django reproduit fidèlement le domaine, ce qui compte pour les requêtes de graphe du lab 1.3. À défaut, **n'importe quel projet que vous connaissez déjà** fonctionne pour les labs 1.1 à 1.3 : ils n'ont besoin que de code et d'un dépôt git.

Passer l'assistant en mode autonome

À partir d'ici, on ne valide plus chaque action une par une : c'est tout l'objet du module.

	GitHub Copilot	Claude Code
En cours de session	`Shift + Tab` bascule le mode de pilotage (jusqu'à **Autopilot**)	`Shift + Tab` fait défiler les modes de permission (jusqu'à **bypass permissions**)
Dès le lancement	`copilot --yolo`	`claude --dangerously-skip-permissions`

> Vous faites ça **sur un dépôt jetable, dans le cadre du lab** : ce n'est pas la configuration à emporter chez un client. Le chapitre 1 des slides explique quels droits vous ouvrez et où ils se déclarent, et le lab 1.2 repose un garde-fou par-dessus.

- **Copilot** : par défaut, il demande une confirmation à chaque appel d'outil et refuse l'action si vous ne répondez pas. Beaucoup de gens répondent alors « oui » machinalement, exactement le comportement qu'on veut éviter : choisissez votre mode consciemment. Créez-vous un alias si vous répétez la même commande.
- **Claude Code** : les règles `deny` de `.claude/settings.json` s'appliquent **même** en mode bypass ; c'est ce que vous exploiterez au lab 1.2.

1.1 · Outils MCP

Objectif : comprendre ce que l'agent sait déjà faire, ouvrir les droits au strict nécessaire, brancher un serveur, **en pilotant l'assistant plutôt qu'en éditant les fichiers à la main**.

Le parti pris : configurer un serveur MCP est typiquement la tâche ingrate pour laquelle on passe vingt minutes à chercher la bonne clé dans la bonne doc. C'est aussi exactement ce qu'un assistant fait bien. On s'en sert donc autrement que pour écrire du code, et vous verrez au passage où il écrit, ce qu'il écrit, et ce qu'il vous fait ouvrir comme droits.

GitHub Copilot : élargir GitHub, brancher un tiers

1. **Constater ce qui est déjà branché**, dans une session `copilot` à la racine de votre codebase :

```text
/mcp show github-mcp-server
```

Repérez `list_issues`, `search_code`, `get_file_contents`. Cherchez `create_issue` et `create_pull_request` : ils sont **absents**, l'écriture n'est pas activée par défaut.

2. **Mesurer le coût du branchement** avant d'ajouter quoi que ce soit : `/context`, et notez la ligne **MCP Tools**. C'est ce que vous payez à **chaque** requête, avant même d'avoir écrit un prompt.
3. **Tester les outils en lecture** : *« Liste les 5 dernières issues ouvertes »*, *« Quelles branches ont eu des commits cette semaine ? »*
4. **Faire ouvrir l'écriture par l'agent, au strict nécessaire.** Demandez-lui, en langage naturel, d'activer **uniquement** le toolset dont vous avez besoin pour créer des issues, et de vous expliquer ce que ça ouvre exactement. Puis demandez-lui ce que coûterait le fait de **tout** activer, et pourquoi ce n'est pas souhaitable. *À observer* : propose-t-il la bonne granularité, ou part-il directement sur l'option qui ouvre tout ? Vous restez le garant du périmètre.
5. **Faire brancher un serveur tiers par l'agent.** Choisissez un serveur qui vous servirait vraiment (`context7` pour la documentation, votre outil de ticketing, un moteur de recherche…) et demandez à l'agent de le configurer **dans le projet**, avec la clé d'API en **variable d'environnement** et non en clair. Exigez trois choses dans votre demande : qu'il vous dise **dans quel fichier** il écrit, qu'il ne mette **aucun secret** dans un fichier versionné, et qu'il vous explique comment vérifier que ça marche.
6. **Relire ce qu'il a produit.** Ouvrez le fichier de configuration, comprenez sa structure, et vérifiez qu'aucune clé n'y traîne. C'est l'étape qui compte : on délègue le geste, pas la responsabilité.
7. **Provoquer et diagnostiquer une panne.** Invalidez la variable d'environnement, relancez la session : le serveur **apparaît toujours dans la liste** mais les appels échouent. Faites diagnostiquer l'erreur par l'agent, puis rétablissez.

Critère de réussite : le serveur tiers répond, aucun secret n'est écrit en clair, vous savez **dans quel fichier** la configuration vit, et vous pouvez chiffrer ce que vos serveurs MCP coûtent en contexte.

> Emplacements de configuration MCP côté Copilot, pour **contrôler** l'agent (pas pour le remplacer) : `~/.copilot/mcp-config.json` (utilisateur, écrit aussi par `copilot mcp add` ou `/mcp add`), `./copilot/mcp-config.json` (projet, versionné, prioritaire), et `--additional-mcp-config` pour charger un fichier à la volée. Piège de copier-coller : VS Code utilise `.vscode/mcp.json` avec la clé `servers` au lieu de `mcpServers`. Pour élargir les droits GitHub en ligne de commande : `copilot --add-github-mcp-toolset <toolset>` (ciblé) et `copilot --enable-all-github-mcp-tools` (tout, **à ne pas garder**).

Claude Code (facultatif) : brancher le MCP GitHub

Lab facultatif : si vous avez déjà branché le MCP GitHub et que le paramétrage est encore en place, ce qui suit est un **rappel**. Si le temps manque, passez directement au 1.2.

1. **Partir de l'état des lieux** : `/mcp` dans la session pour voir ce qui est branché. Contrairement à Copilot, **rien n'est pré-installé** côté Claude Code : c'est vous qui décidez de tout.
2. **Mesurer le coût du branchement** avant d'ajouter quoi que ce soit : `/context`, et notez la ligne des outils MCP. C'est ce que vous paierez à **chaque** requête.
3. **Faire brancher le serveur GitHub par l'agent.** Demandez-lui, en langage naturel, de configurer le serveur MCP GitHub **pour ce projet**, avec un jeton en **variable d'environnement** et non en clair. Exigez les trois mêmes choses : le fichier où il écrit, aucun secret dans un fichier versionné, comment vérifier que ça marche.
4. **Relire ce qu'il a produit.** Ouvrez le fichier de configuration, comprenez sa structure, vérifiez qu'aucun jeton n'y traîne.
5. **Interroger le périmètre.** Demandez à l'agent quels droits porte le jeton qu'il vous a fait créer, et ce qu'il pourrait faire sur vos dépôts avec. Réduisez si c'est trop large : c'est le réflexe **scope minimal** du chapitre 1.
6. **Tester en lecture** : *« Liste les 5 dernières issues ouvertes »*, *« Quelles branches ont eu des commits cette semaine ? »*
7. **Provoquer et diagnostiquer une panne.** Invalidez le jeton, relancez : le serveur **apparaît toujours** dans `/mcp` mais les appels échouent. Faites diagnostiquer l'erreur par l'agent, puis rétablissez. Relancez `/context` : un serveur **non fonctionnel** coûte quand même ses tokens de déclaration.

Critère de réussite : l'agent interroge GitHub via MCP, aucun secret n'est écrit en clair, vous savez **dans quel fichier** la configuration vit et **quels droits** porte votre jeton, et vous savez diagnostiquer un jeton manquant ou invalide.

> Pour contrôler l'agent : la configuration projet vit dans un `.mcp.json` à la racine (versionné, partagé par l'équipe) ; `claude mcp add` écrit la même chose depuis la ligne de commande.

Ce qui ne bouge pas d'un harness à l'autre

> **L'asymétrie vaut d'être notée** : côté Copilot, GitHub est déjà branché en lecture seule et l'exercice consiste à **restreindre** ce qui est ouvert ; côté Claude Code, rien n'est là et l'exercice consiste à **n'ouvrir que le nécessaire**. Même règle, gestes opposés. Le geste à retenir n'est pas « brancher un MCP » mais **« ouvrir le minimum nécessaire »** : un serveur en écriture totale, c'est un agent qui peut créer des PR sur vos dépôts sans que vous l'ayez demandé.

Pourquoi on ne fait plus ce lab à la main : les chemins de configuration et les noms d'options bougent d'une version à l'autre, et les recopier depuis une fiche apprend surtout à copier. Ce qui reste vrai six mois plus tard, c'est le **réflexe** : demander, relire, vérifier le périmètre.

1.2 · Activer un hook local

Objectif : poser un hook dans le projet, le voir agir dans la session, puis le faire **refuser** une commande : la démonstration que « la policy vit dans le runtime, pas dans le prompt ».

GitHub Copilot

1. **Déclarer un hook** dans `.github/hooks/lab-hooks.json` à la racine du projet (fichier **versionné**) :

```json
{
  "hooks": {
    "sessionStart": [
      {
        "type": "command",
        "bash": "cat > /dev/null; echo '{\"additionalContext\":\"Message de la formation AIDD
        "timeoutSec": 10
      }
    ]
  }
}
```

Ce hook **n'affiche rien** : la sortie brute d'un hook est ignorée par la CLI. Son seul canal est `additionalContext`, qui injecte du texte dans le contexte du modèle : on ne verra pas le message, on verra son **effet**. Le `cat > /dev/null` consomme l'événement que Copilot passe en JSON sur stdin ; un script qui ne le lit pas peut bloquer. (Le « mot de passe » est une valeur fictive du lab.)

2. **Recharger** : quittez la session et relancez `copilot`. Les hooks sont lus **au démarrage**, il n'y a pas de rechargement à chaud.
3. **Le voir se déclencher** : demandez *« quel est le mot de passe du lab ? »*. L'agent répond **BANANE42**, une information qui n'est ni dans le dépôt, ni dans votre prompt : elle ne peut venir que du hook.
4. **Passer de l'injection de contexte au blocage.** Créez `.github/hooks/no-rm.sh`, puis `chmod +x .github/hooks/no-rm.sh` :

```bash
#!/usr/bin/env bash
# Copilot passe l'appel d'outil en JSON sur stdin :
# {"sessionId":…,"toolName":"bash","toolArgs":"{\"command\":\"rm -f …\"}"}
payload=$(cat)
if echo "$payload" | grep -qE '\brm\b'; then
  echo '{"permissionDecision":"deny","permissionDecisionReason":"Interdit pendant le lab : su
fi
exit 0    # ne rien émettre = laisser passer
```

Ajoutez-le au même fichier de configuration, à côté du `sessionStart` :

```json
"preToolUse": [
  { "type": "command", "matcher": "bash",
    "bash": "$COPILOT_PROJECT_DIR/.github/hooks/no-rm.sh", "timeoutSec": 10 }
]
```

`$COPILOT_PROJECT_DIR` pointe la racine du projet : **jamais de chemin absolu en dur** dans un fichier versionné, il ne fonctionnerait que sur votre machine. Le `matcher` est une regex sur le **nom de l'outil** (`bash` , `view` , `glob` , `edit` …), compilée en `^(?:…)$` ; sans `matcher` , le hook se déclenche sur tous les appels.

5. **Vérifier le refus** : relancez la session, créez un fichier jetable (`touch /tmp/lab-hook.txt`) et demandez « *supprime le fichier /tmp/lab-hook.txt avec rm* ». La CLI affiche le refus sous l'appel d'outil, et l'agent l'annonce dans sa réponse :

```text
✗ Remove lab-hook.txt file (shell)
  │ rm -f /tmp/lab-hook.txt && echo done
  └ Denied by preToolUse hook: Interdit pendant le lab : suppression de fichiers via rm.
```

Vérifiez que le fichier est **toujours là**, et refusez les contournements que l'agent proposera spontanément.

Critère de réussite : l'agent restitue le mot de passe injecté par le `sessionStart` , et le `rm` est refusé sans que le modèle ait son mot à dire.

> **Les hooks de dépôt ne se déclenchent pas en mode non interactif** (`copilot -p "…"`), comportement constaté sur les versions de CLI de mi-2026, à revérifier le jour J. Ce lab se fait donc en session interactive. Pour un hook scripté en CI, posez-le en portée utilisateur (`~/.copilot/hooks/`), qui reste actif en `-p` .

> Emplacements : `.github/hooks/*.json` (dépôt, versionné, **également lu par l'agent cloud**) et `~/.copilot/hooks/` (utilisateur, `%USERPROFILE%\.copilot\hooks` sous Windows). Alternative à `$COPILOT_PROJECT_DIR` : le champ `cwd` de l'entrée, résolu depuis la racine du dépôt (`"cwd": ".github/hooks", "bash": "./no-rm.sh"`). Trois types de hooks : `command` (champs `bash` / `powershell` / `command` selon la plateforme), `http` (POST vers un endpoint, pratique pour la télémétrie d'entreprise) et `prompt` (soumet automatiquement un texte, **uniquement** sur `sessionStart`). Les 14 événements sont listés dans `docs.github.com/copilot/reference/hooks-reference` .

> Règles de sortie d'un hook `command` : le stdout n'est jamais montré à l'utilisateur, seul le **dernier objet JSON** est interprété (deux objets JSON se concatènent en JSON invalide et sont ignorés en silence). Côté `preToolUse`, `permissionDecision` vaut `allow` / `deny` / `ask` (`permissionDecisionReason` obligatoire pour `deny`), et un **code de sortie non nul refuse aussi l'appel** (fail-closed), mais sans message maîtrisé. Un timeout, à l'inverse, laisse passer (fail-open).

Claude Code

1. **Déclarer un hook** dans `.claude/settings.json` (à la racine du projet) :

```json
{
  "hooks": {
    "SessionStart": [
      { "hooks": [{ "type": "command", "command": "echo '{\"systemMessage\": \"Bienvenue dans
    ]
  }
}
```

Sans le JSON `systemMessage`, aucun message n'apparaît : **les sorties standard ne sont jamais affichées par Claude**, sauf en mode debug.

2. **Recharger** : quittez la session et relancez `claude`. Les hooks sont capturés **au démarrage** : éditer `settings.json` en cours de session ne suffit pas. Vérifiez avec `/hooks` que les entrées sont bien enregistrées.
3. **Le voir se déclencher** : au lancement, vous devez voir le message `SessionStart:startup says: Bienvenue dans cette formation`.
4. **Passer de l'observation au blocage.** Créez `.claude/hooks/no-rm.sh`, puis `chmod +x .claude/hooks/no-rm.sh` :

```bash
#!/usr/bin/env bash
# Claude Code passe l'appel d'outil en JSON sur stdin
payload=$(cat)
if echo "$payload" | grep -qE '"command"[[:space:]]*:[[:space:]]*"[^"]*\brm\b'; then
  echo "Interdit pendant le lab : suppression de fichiers via rm." >&2
  exit 2    # 2 = refuser l'appel ; stderr est renvoyé au modèle
fi
exit 0
```

Branchez-le sur `Bash`, à côté du `echo` existant :

```json
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash", "hooks": [{ "type": "command", "command": "$CLAUDE_PROJECT_DIR/.cl
    ]
  }
}
```

5. **Vérifier le refus** : relancez la session, créez un fichier jetable (`touch /tmp/lab-hook.txt`) et demandez « *supprime le fichier /tmp/lab-hook.txt* ». L'agent tente le `rm` , le hook le refuse, et **l'agent annonce le refus** avec la raison renvoyée par le script, souvent en proposant un contournement, qu'il faut refuser aussi.

Critère de réussite : les deux hooks apparaissent dans `/hooks` , leur sortie est visible dans la transcription, et le `rm` est refusé sans que le modèle ait son mot à dire.

> Emplacements : `.claude/settings.json` (projet, versionné, celui du lab), `.claude/settings.local.json` (projet, non versionné) et `~/.claude/settings.json` (utilisateur). C'est le **code de sortie** qui porte la décision d'un `PreToolUse` : `0` laisse passer, `2` refuse l'appel et renvoie stderr au modèle, tout autre code signale une erreur sans bloquer. Pour un contrôle plus fin, le hook peut écrire du JSON sur stdout (`hookSpecificOutput.permissionDecision` à `allow` / `deny` / `ask`).

Dernière étape, commune : la même règle en langage naturel

Écrivez « *ne supprime jamais de fichier avec rm* » dans `AGENTS.md` (Copilot) ou `CLAUDE.md` (Claude Code), retirez le hook, redemandez la suppression. Une consigne est une **suggestion** ; un hook est une **garantie**.

> Équipe mixte ? Copilot accepte **aussi la syntaxe Claude Code** dans ses propres fichiers (`"PreToolUse"` avec un tableau d'entrées `matcher` + `hooks`) et il lit les hooks déclarés dans `.claude/settings.json` . L'inverse n'est pas vrai : Claude Code ne lit pas `.github/hooks/` . Une équipe qui mélange les deux harness a donc intérêt à écrire ses hooks partagés au format Claude Code, et à réserver `.github/hooks/` à ce qui doit tourner côté agent cloud.

1.3 · Indexer sa codebase

Objectif : Graphify installé, knowledge graph de la PetClinic construit, requêtes sémantiques fonctionnelles.

1. **Installer et lancer Graphify** sur le projet, en **nommant la plateforme** :

GitHub Copilot	Claude Code
`uv tool install graphifyy` (ou `pip install graphifyy`)	`uv tool install graphifyy` (ou `pip install graphifyy`)
`graphify install --platform copilot` : dépose le `SKILL.md` pour Copilot	`graphify install --platform claude` : enregistre le skill dans Claude Code
`graphify .` : construit `graphify-out/`	dans Claude Code : `/graphify`

Résultat : `graphify-out/graph.html` , `GRAPH_REPORT.md` , `graph.json` .

> `--platform` **n'est pas optionnel dans la vraie vie.** Sans lui, Graphify **détecte** les agents installés sur votre machine et choisit tout seul : sur un poste où plusieurs assistants cohabitent, le skill peut atterrir chez le mauvais (par exemple dans `.claude/skills/` alors que vous travaillez sous Copilot), et l'agent ne le trouvera jamais. C'est le piège le plus fréquent de ce lab, dans les deux sens.

2. **Copilot : vérifier où le skill a été écrit**, et le déplacer si nécessaire. Copilot lit les trois emplacements ; un skill est un **simple dossier** contenant un `SKILL.md`, le déplacer d'un agent à l'autre est un `mv`.

```bash
ls .github/skills/ .claude/skills/ .agents/skills/ 2>/dev/null
# si le skill est ailleurs, il suffit de déplacer son dossier :
# mv <dossier-du-skill> .github/skills/
```

3. **Faire en sorte que le graphe reste à jour**, sinon il périme dès le premier commit. C'est de l'analyse **AST pure** : aucun appel de modèle, donc **aucun coût en tokens**, juste du CPU.

```bash
graphify hook install     # hooks git post-commit + post-checkout : rebuild automatique
graphify . --update       # rafraîchissement manuel : ne réextrait que les fichiers modifiés
```

4. **Prendre un tour d'horizon** : demandez à l'agent de lire `graphify-out/GRAPH_REPORT.md` et de résumer l'architecture de la PetClinic.
5. **Tester les requêtes**, rien d'autre à lancer, le skill suffit :

GitHub Copilot (langage naturel)	Claude Code
« Quelles classes implémentent OwnerRepository ? »	`/graphify query "quelles classes implémentent OwnerRepository ?"`
« Quel est le chemin de dépendances entre OwnerController et VisitRepository ? »	`/graphify path "OwnerController" "VisitRepository"`
« Liste les classes sans tests dans `src/main/java/` »	même question en langage naturel, sans nommer le skill

Observez dans les appels d'outils que l'agent exécute `graphify query …` / `graphify path …` plutôt que d'ouvrir les fichiers un par un.

6. **Copilot : chiffrer le gain.** Lancez `/context` avant puis après une série de questions sur l'architecture : une fois en laissant l'agent utiliser le graphe, une fois (`/clear`) en lui demandant explicitement de répondre **en lisant les fichiers**. Comparez la ligne **Messages**.

Critère de réussite : l'agent répond aux questions de relations **en interrogeant le graphe** (commandes `graphify query` / `graphify path` dans les appels d'outils), et, côté Copilot, le poste **Messages** de `/context` est nettement plus bas qu'avec des lectures fichier par fichier.

> Les exemples sont ceux de la PetClinic Java (`src/main/java/` , `OwnerRepository` , `OwnerController` …). Sur la variante Python ou sur votre propre projet, transposez : les questions de relations (« qui appelle quoi ? », « quelles implémentations de telle interface ? », « quels modules sans tests ? ») se posent à l'identique quel que soit le langage.

> **Aucun serveur MCP n'est nécessaire ici.** Graphify s'utilise comme un skill : le `SKILL.md` déposé par `graphify install` décrit à l'agent les commandes à lancer (`graphify query` , `graphify path` , `graphify explain`), et l'agent les exécute lui-même via le shell. Un serveur MCP Graphify existe, mais il ne sert qu'à exposer ces capacités à un outil qui ne saurait pas lire un skill.

Bouclage avec le lab 1.2 : sans incitation, l'agent retombe volontiers dans ses habitudes de lecture directe, et vous ne verrez jamais `graphify` dans les appels d'outils.

- **Copilot** : ajoutez vous-même un hook `preToolUse` dans `.github/hooks/` sur le matcher `view|glob` , qui lui rappelle que `graphify query` existe. Ce n'est pas optionnel.
- **Claude Code** : `graphify install` configure automatiquement un hook `PreToolUse` qui pousse l'agent vers le graphe avant chaque lecture de fichier. Ouvrez `.claude/settings.json` et comparez-le avec les hooks que vous avez écrits à la main.

> Deux notions de « hook » se croisent dans ce lab : `graphify hook install` pose des hooks **git** (post-commit / post-checkout) qui reconstruisent le graphe quand le code change ; le `preToolUse` est un hook **d'agent**, qui s'exécute quand l'assistant s'apprête à lire un fichier. Même mot, deux mécanismes sans rapport.

> **Inutile de committer** `graphify-out/` . C'est un artefact dérivé du code, comme un dossier de build : il se régénère en quelques secondes, un graphe périmé fait raisonner l'agent sur un état qui n'existe plus, et il produit des conflits garantis dès qu'on travaille sur plusieurs branches. Le partage en équipe passe par le hook git que chacun installe. Ajoutez `graphify-out/` à votre `.gitignore` .

Labs inline des slides (Ch. 2)

Mesurer sa context window

Mesurez deux fois, à froid puis après 20 minutes de travail : `/context` (Copilot CLI, Claude Code), `/status` (Codex, plus l'indicateur de remplissage en %). Quel poste a bougé ? Lequel était déjà lourd avant la première question (instructions custom, outils MCP) ?

Vérifier sa mémoire

1. **Vérifier** qu'elle est activée, pour pouvoir relire après les deux jours ce que l'agent a retenu, et ce qu'il a mal retenu.
2. **Localiser** : `/memory` (Claude Code comme Copilot CLI). Vous pouvez aussi demander à Copilot si sa mémoire est activée et où il la stocke, puis aller consulter.
3. **Écrire** : donnez-lui un fait à retenir (une préférence de style, un surnom).

4. **Vérifier** : nouvelle session, redemandez-le sans le rappeler. S’il répond, la mémoire a traversé la session.

SUPPORT D'EXERCICE

Lab 2 · User stories

Deux temps sur la PetClinic : brainstormer votre définition d'une bonne spec et l'encoder en skill (2.1), puis vous en servir pour générer une user story (2.2).

2.1 · Votre définition d'une bonne spec

Objectif : ne pas *recevoir* un guide tout fait, mais **brainstormer le vôtre, puis l'encoder en skill**. Concentrez-vous sur le **fond** plus que sur la forme.

1. **Brainstorm d'équipe** : *« c'est quoi une bonne spec pour nous ? »*
   - quel **gabarit** (story, critères d'acceptation, screenshots, **Definition of Done**) ?
   - quelle **syntaxe** (*As a… I want… so that…*), quel niveau de **détail** ?
   - **où vit-elle ?** (`docs/stories` , PRD, ticket Jira…) Quelle **source de vérité** unique ?
2. **Encoder** le consensus dans un skill `story-writing` (`SKILL.md` + gabarit), **versionné** dans le dépôt : `.github/skills/story-writing/` côté Copilot, `.claude/skills/story-writing/` côté Claude Code.
3. **Vérifier qu'il se déclenche** : demandez *« écris-moi une user story pour … »* dans une session neuve, sans nommer le skill. S'il ne s'active pas, c'est la `description` qu'il faut retravailler, pas le corps.

Plusieurs manières d'écrire le skill, comparez-les si vous avez le temps :

Manière	GitHub Copilot	Claude Code
À la main	vous créez le dossier et le `SKILL.md` vous-même	idem
En pilotant le CLI	vous demandez à `copilot` de créer le skill à partir du consensus de l'étape 1	vous demandez à l'agent de le créer : il connaît le format, renseigne le frontmatter et vérifie que le skill est bien formé
Assistant guidé	VS Code : la palette de commandes propose une **création de skill/agent** qui vous questionne au fur et à mesure (nom, description, contexte, outils autorisés) et vérifie le format. **Absent du CLI** : l'un des écarts les plus visibles entre les deux surfaces de Copilot	—

> Vous redevenez **garants** de ce que « bonne spec » veut dire, au lieu de copier-coller un prompt.

> L'assistant guidé (ou l'agent) est utile pour un **premier** skill : il évite les oublis de frontmatter et pose les bonnes questions. Une fois le format acquis, écrire le fichier directement va plus vite.

> Sur Copilot, un skill n'est **pas** une commande : il est chargé par le modèle quand sa `description` correspond à la tâche. C'est donc la ligne la plus importante du fichier. Pour un déclenchement **explicite** par l'utilisateur, créez plutôt un prompt file (`.github/prompts/<nom>.prompt.md` , invoqué par `/<nom>` dans l'IDE) ou un agent custom appelé par `/agent` .

2.2 · Rédiger une user story

Objectif : générer une user story en suivant **le guide que vous venez d'écrire**, sur la PetClinic.

1. Pointez l'IA vers **votre skill** `story-writing` (ou `docs/product-management/writing-guides/story-writing-guides.md`) ; relisez-le une dernière fois.
2. Demandez à l'IA de **suivre ce guide** pour écrire la story d'une nouvelle page **« All Clinics »** :
   - **Clinic** devient une **entité** (aujourd'hui un champ de **Visit**), en **lecture seule** ;
   - relation **ManyToOne** Visit → Clinic ; `Clinic { id, address, director }` ;
   - **director est un Vet** → relation **OneToOne** Clinic → Director ;
   - pas de migration de base (pas encore en prod).
3. Générez la story dans un fichier `.md` , relisez-la et enregistrez-la dans `docs/stories` .

> Une story cadrée par *votre* guide est **concise, dans votre syntaxe, avec une Definition of Done**. Elle resservira au lab 8 (Feature A, page « All Clinics »).

SUPPORT D'EXERCICE

Lab 3 · Pomodoro

Spécifier puis générer une petite application Pomodoro (HTML + JavaScript, côté client), d'abord avec OpenSpec (3.1), puis avec GitHub Spec Kit (3.2).

3.1 · Construire un Pomodoro avec OpenSpec

Objectif : spécifier puis générer une petite app **Pomodoro** (HTML + JS, côté client) avec **OpenSpec**.

1. Créez un dossier `Pomodoro`, placez-vous dedans (côté Claude Code : ouvrez-y Claude), puis installez et initialisez OpenSpec :

```bash
npm install -g @fission-ai/openspec@latest
openspec init --tools github-copilot    # GitHub Copilot
openspec init --tools claude            # Claude Code
```

2. **Proposer** : lancez le skill `openspec-propose` avec le besoin : **25 min de focus / 10 min de pause** par défaut ; *(avancé)* boutons **pause** et **reset** ; *(avancé)* un **design « Chupa Chups »**. Invitez l'IA à proposer des features.
3. **Appliquer** : une fois les specs définies, lancez le skill `openspec-apply-change` avec le nom du changement, puis **testez** l'application.
4. **Archiver** : si vous êtes satisfait, lancez le skill `openspec-archive-change` pour archiver les specs.
5. **Extension** : ajoutez un **bouton de bascule** entre mode *Chupa Chups* et mode *Coca-Cola*.

Skill OpenSpec	GitHub Copilot	Claude Code
`openspec-propose`	`/openspec-propose`	`/opsx:propose`
`openspec-apply-change`	`/openspec-apply-change <nom-du-changement>`	`/opsx:apply <nom-du-changement>`
`openspec-archive-change`	`/openspec-archive-change <nom-du-changement>`	`/opsx:archive <nom-du-changement>`

> On a fait **spec d'abord, code ensuite**, et la spec reste la **source de vérité** régénérable.

3.2 · Construire le Pomodoro avec Spec Kit

Objectif : refaire le **Pomodoro** avec **GitHub Spec Kit**, en gardant la **charte visuelle** de l'étape précédente et en parcourant **tout le cycle** Spec Kit.

```bash
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
specify init Pomodoro-SpecKit --integration copilot   # GitHub Copilot
specify init Pomodoro-SpecKit                         # Claude Code : choisir l'assistant + l
```

1. **Copilot : observer ce que l'init a écrit.** `git status` montre les commandes `/speckit.*` déposées dans les dossiers que Copilot lit. Ce sont des fichiers versionnés, pas une dépendance externe.
2. Déroulez les **4 étapes** : `/speckit.specify` → `/speckit.plan` → `/speckit.tasks` → `/speckit.implement`.
3. **Réutilisez** la **charte « Chupa Chups »** produite avec OpenSpec : reprenez la spec ou les fichiers, ou **injectez une capture d'écran** au moment du `specify`.
4. Faites du **mode « Coca-Cola »** une **nouvelle feature** : relancez un `/speckit.specify` dédié, puis `plan` / `tasks` / `implement`.

> **À vous de jouer** : plusieurs chemins fonctionnent.

> Spec Kit est un projet GitHub : l'intégration Copilot est de première classe. Les mêmes 30+ intégrations restent disponibles (`--integration claude` , `cursor` , `gemini` ...) : le TP est identique, seul le flag change.

SUPPORT D'EXERCICE

Lab 4 · Extension speckit-doc-tracker

Créer une extension Spec Kit `speckit-doc-tracker` qui enrichit chaque commit d'un indicateur de part de code générée par l'IA (4.1, 4.2) et vérifie que `SPEC.md` est à jour (4.3). Identique pour GitHub Copilot et Claude Code : prérequis, un projet déjà initialisé par `specify init` (lab 3.2).

4.1 · Créer et installer l'extension

Objectif : créer une extension Spec Kit `speckit-doc-tracker` qui enrichit chaque commit d'un **indicateur de part de code généré par l'IA** et d'une vérification de `SPEC.md` .

1. **Créer le projet d'extension** :

```text
speckit-doc-tracker/
├── extension.yml        # manifeste : nom, description, commandes
└── commands/
    └── commit.md        # skill : logique de /speckit.doc-tracker.commit
```

Dans `extension.yml` :

```yaml
schema_version: '1.0'
extension:
  id: doc-tracker
  name: 'Documentation Tracker'
  version: '0.0.1'
  description: 'Tracks documentation of projects in SPEC.md'

requires:
  speckit_version: '≥0.9.0'

provides:
  commands:
    - name: speckit.doc-tracker.commit
      file: commands/commit.md
      description: 'Writes commit messages, updates documentation on each commit.'
```

2. **Installer en local** pour itérer :

```bash
specify extension add --dev ./speckit-doc-tracker
specify extension list        # vérifier qu'elle est active
```

> Prendre l'habitude de committer **via l'assistant de code** (et non en ligne de commande directe) est ce qui rend la traçabilité possible : l'assistant connaît le **contexte complet** du diff. Un `git commit -m "..."` direct court-circuite tous les contrôles ; la valeur est dans le flux complet : diff → analyse → message → vérification.

> L'extension est d'abord développée et versionnée à part, puis ses artefacts installés sont versionnés dans le dépôt du projet, comme n'importe quel fichier de config.

4.2 · La commande de commit enrichie

1. **Écrire la skill** `commands/commit.md` : elle demande à l'agent d'analyser les fichiers stagés, d'estimer la part IA, puis de générer un commit conventionnel avec le footer dédié :

```text
feat(auth): add refresh token endpoint

Implements JWT refresh flow with 7-day expiry.

AI-Generated: ~75%
```

2. **Tester** en invoquant la commande :

```text
/speckit.doc-tracker.commit
```

L'agent lit le diff stagé, rédige le message, ajoute le footer `AI-Generated`.

> L'estimation de la part IA est **approximative par nature** : ce qui compte, c'est d'établir une **convention d'équipe cohérente** et de la rendre visible dans l'historique git.

4.3 · SPEC.md à jour avant chaque commit

1. **Ajouter le format de** `SPEC.md` dans `templates/SPEC.md`. Dans les étapes suivantes, l'extension peut faire référence à ce fichier.

```markdown
# [Project Name] Specification

Purpose: [One sentence describing what the project provides and to whom.]

## Normative Language

The key words MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL in

Implementation-defined means the behavior is part of the implementation contract, but this sp

## §1 Problem Statement

## §2 Goals & Non-Goals

## §3 System Overview

## §4 Domain Model

## §5 Functional Behavior

## §6 Technical Architecture

## §7 Glossary

## §8 References
```

2. **Enrichir la commande** : avant de finaliser le commit, l'extension **détecte si** `SPEC.md` **doit être mis à jour** au regard du diff stagé :

Situation	Action de l'extension
Nouveau endpoint / composant ajouté	Vérifie que §5 (Functional Behavior) de `SPEC.md` reflète le changement
Modification d'architecture	Vérifie que §3 / §6 (System Overview / Technical Architecture) sont à jour
`SPEC.md` non modifié depuis N commits	Suggère de lancer `/speckit.analyze` pour auditer la couverture

3. **En cas de drift détecté** : l'agent **propose de lancer** `/specify` **ou** `/plan` pour mettre `SPEC.md` à jour, puis de **reprendre le commit** une fois fait.
4. **Fil rouge** : l'extension sera installée dès le départ du **TP de pilotage E2E** (lab 5) ; chaque commit vérifiera que `SPEC.md` documente les endpoints et que les **logs applicatifs** (structurés, avec niveau et contexte) sont décrits dans la spec (§5 Functional Behavior).

> Le commit devient un **point de synchronisation** entre le code et la spec : ni contraignant, ni silencieux. Le principe est « guider puis bloquer » : l'agent ne rejette pas brutalement, il montre ce qui dérive et propose de le corriger via la commande Spec Kit appropriée, à la différence d'un pre-commit hook rigide.

Variante « ceinture et bretelles » : la commande **guide**, mais si l'équipe veut une règle **non contournable**, on double le contrôle par un hook qui refuse un `git commit` lancé directement (côté Copilot : un `preToolUse` dans `.github/hooks/` ; côté Claude Code : un `PreToolUse` sur `Bash`, comme au lab 1.2). C'est le principe « la policy vit dans le runtime, pas dans le prompt ».

SUPPORT D'EXERCICE

Lab 4.4 à 4.6 · Phase de design (facultatif)

Labs facultatifs : Spec Kit enchaîne `specify → plan` sans étape de conception ; on en insère une avec une extension, un deuxième exemple après `doc-tracker`. Variante visuelle (wireframe, 4.4 puis 4.5) **ou** variante backend (contrat d'API, 4.6), selon le groupe. Identique pour GitHub Copilot et Claude Code.

4.4 · Ajouter une phase de design

Objectif : étendre Spec Kit pour intercaler un **wireframe basse fidélité (HTML)** entre `specify` et `plan`.

1. Créez un **projet d'extension** Spec Kit :
   - manifeste `extension.yml` + `commands/wireframe.md` → commande `/speckit.ext.wireframe` ;
   - la commande **génère un wireframe HTML basse fidélité** à partir de la spec, **avant** le plan.
2. **Installez en local** pour itérer :

```bash
specify extension add --dev ./speckit-wireframe
specify extension list        # vérifier qu'elle est active
```

> On insère une **étape maison** dans le pipeline : la customisation en action.

4.5 · Rejouer le Pomodoro, puis comparer

1. **Rejouez** la création du **Pomodoro** en intégrant la nouvelle phase. Le **wireframe** fixe le **résultat attendu** : on contrôle finement le design **avant** l'implémentation.

```text
/speckit.specify → /speckit.ext.wireframe → /speckit.plan → /speckit.tasks → /speckit.impleme
```

2. **Comparez** en fin d'implémentation : confrontez le **design réel** de l'app au **wireframe**. Écarts visuels ? Intention respectée ? Faut-il **raffiner la spec** ou **la phase wireframe** ?

> La phase de design devient un **garde-fou réutilisable**, et le wireframe, un critère de recette.

4.6 · Variante backend : une phase de design de contrat API

Objectif : même mécanique d'extension (insérer une étape de conception avant l'implémentation), sur un terrain **backend** : concevoir le **contrat OpenAPI** d'un nouvel endpoint PetClinic avant de l'implémenter, plutôt qu'un wireframe HTML.

1. Créez un **projet d'extension** Spec Kit :
   - manifeste `extension.yml` + `commands/api-contract.md` → commande `/speckit.ext.api-contract` ;
   - la commande **génère ou met à jour un fragment OpenAPI** (`openapi.yaml`) à partir de la spec, **avant** le plan.
2. **Installez en local** pour itérer :

```bash
specify extension add --dev ./speckit-api-contract
specify extension list        # vérifier qu'elle est active
```

3. **Déroulez** sur un nouvel endpoint PetClinic (ex. `GET /owners/{id}/pets/count`). Le **contrat OpenAPI** fixe le **résultat attendu** (chemins, schémas, codes retour) : on contrôle finement l'interface **avant** l'implémentation.

```text
/speckit.specify → /speckit.ext.api-contract → /speckit.plan → /speckit.tasks → /speckit.impl
```

4. **Comparez** en fin d'implémentation : l'API réelle respecte-t-elle le contrat généré ? Écarts de schéma ? Faut-il **raffiner la spec** ou **le contrat** ?

> Même point pédagogique que la variante wireframe : fixer le contrat avant le code donne un critère de recette objectif, adapté à une équipe qui pilote par les contrats plutôt que par le rendu visuel.

SUPPORT D'EXERCICE

Lab 5 · Pilotage des tests en Spec-Driven Development

Construire, en Spec-Driven Development, une application qui pilote des tests E2E dans un navigateur (5.2) ou, en variante backend, des tests de contrat d'API (5.3), et repartir avec un produit utilisable. Lab long : l'objectif n'est pas de « finir ». Identique pour GitHub Copilot et Claude Code.

5.1 · Le parti pris du TP

- **Dev totalement automatisé** : le **code est un asset régénérable**, on ne s'en préoccupe pas **tant que la spécification est respectée**.
- **Qualité et maintenabilité** doivent être **au rendez-vous** : c'est tout l'objet du framework.
- **Sécurité dès le départ** : vous pouvez imposer l'**authentification de tous les points d'accès**, les secrets hors du code…
- **Architecture minimale** à ce stade : un projet **Docker / docker-compose**, une base **type SQLite**… et on **pilote les décisions** (exécution des requêtes, **navigateur headless**…).
- **Clé d'API LLM nécessaire** : c'est l'**application que vous construisez** qui appelle un modèle, pas votre agent. **Votre propre clé d'API** si vous en avez une (l'option la plus réaliste) ; sinon **Google AI Studio (Gemini)**, le chemin le plus court, ou un compte **OpenRouter** (détail dans le handout « Prérequis des labs »). Sur les modèles gratuits, les données partent souvent hors UE : aucune donnée sensible.
- `doc-tracker` **branché dès le départ** : l'extension construite au lab 4 est **installée sur ce projet** ; chaque commit vérifiera que la doc est à jour et tracera la part d'IA.

> On code l'**intention**, pas les lignes : le **moteur s'occupe du reste**.

> « Pourquoi une clé d'API alors qu'on a une licence Copilot ou un abonnement Claude ? » Parce que ce sont deux choses différentes : la licence sert à **développer**, la clé d'API sert au **produit livré**, qui doit interroger un modèle en autonomie pour diagnostiquer un échec de test.

	Tests E2E (5.2)	Tests de contrat d'API (5.3)
Générer	un script de navigation (Playwright), depuis une description	un test de contrat, depuis la spec OpenAPI
Exécuter	captures d'écran, traces	requêtes / réponses tracées, écarts de schéma
Diagnostiquer	vraie régression, ou évolution d'UI ?	régression de contrat, ou évolution assumée ?
Moteur	Playwright	Schemathesis, Pact

Point de départ : la branche `tp_e2e_speckit` du dépôt de TP, initialisée avec Spec Kit, avec un `SPEC.md` rétro-généré depuis le code et deux extensions installées (`agent-context` , `documentation`) ; `speckit-doc-tracker` devient la troisième. L'autre branche, `tp_e2e_framework` , repose sur OpenSpec, dont le mécanisme de personnalisation (schemas) diffère : l'extension `doc-tracker` est propre à Spec Kit et n'y est pas installable telle quelle.

5.2 · Specs du framework (tests E2E)

1. **Décrire** un nouveau test E2E en **langage naturel** → l'IA **génère le script** Playwright correspondant.
2. **Bouton « Exécuter »** dans l'interface → lance le test et **affiche les captures d'écran** prouvant le **succès ou l'échec**.
3. **Historique des runs** : liste des derniers passages avec leur **statut**, pour le **suivi**.

Maquette de l'écran principal (« Pilote E2E : tests ») : à gauche, la liste des **tests** (Connexion, Ajouter un vétérinaire, Réserver une visite, Paiement) et un bouton « + Nouveau test » ; au centre, pour le test sélectionné, la **description en langage naturel**, le **script généré (Playwright)** et le bouton **« Exécuter »** ; en dessous, les **dernières captures** ; à droite, l'**historique des runs** (#13 · 14:02, #12 · 11:30, #11 · hier…).

4. **Consulter le rapport** :
   - **succès** → afficher les **statistiques** (durée, étapes, captures) ;
   - **échec** → **outil d'analyse de cause** assisté par IA :
     - **vraie régression** (la page ne respecte plus l'intention) → **remonter le bug** ;
     - **simple évolution d'UI** (le front a changé, l'intention tient) → l'IA **met à jour le script automatiquement**.

Maquette du rapport (« Rapport : Run #13 », statut **ÉCHEC**) : les **captures** côte à côte, « attendu » et « obtenu » ; un encart **« Analyse de cause (IA) »** avec la cause probable (par exemple « évolution d'UI : libellé du bouton modifié ») et deux issues, **Régression réelle** (la page ne respecte plus l'intention → ouvrir un bug) ou **Évolution d'UI** (l'intention tient → mise à jour du script), avec un bouton **« Réparer (auto) »**. Si succès : statistiques (durée, étapes, captures).

5. **Pas de CI ici**, mais notez que ce run pourrait être **planifié en CI** (à chaque PR, la nuit).

Pendant la génération, vérifiez commit par commit via l'extension `doc-tracker` (labs 4.1 à 4.3) :

1. **Doc à jour** : les endpoints du serveur E2E sont-ils couverts dans `SPEC.md` (§5 Functional Behavior) ?
2. **Logs lisibles** : le serveur émet-il des logs structurés (niveau, message, contexte) sur les actions clés (démarrage d'un run, échec, réparation) ?
3. **Traçabilité IA** : chaque commit indique-t-il la part de code générée automatiquement ?

> La description (l'intention) est la **source** ; le script n'est qu'un **artefact généré**. Le test redevient un **actif vivant** ; le commit, un **point de contrôle de la qualité documentaire et applicative**.

> Sur un TP long comme celui-ci, pensez à `/context` régulièrement, et à `/compact focus on <sujet>` plutôt qu'à un `/compact` sec : le résumé conserve alors ce qui compte pour la suite. Côté Copilot, `/session checkpoints` permet de relire ce que l'agent a retenu après une compaction, au lieu de le supposer.

5.3 · Variante backend : pilotage de tests de contrat

Objectif : même apprentissage, sur un terrain **backend** : piloter des **tests de contrat API** (OpenAPI) décrits en langage naturel, plutôt qu'un navigateur automatisé. À proposer au choix (front ou backend) selon la composition du groupe, pas en remplacement systématique.

1. **Récupérer ou générer la spec OpenAPI** du serveur PetClinic (`openapi.yaml`) ; si elle n'existe pas encore, demandez à l'agent de la générer depuis le code des contrôleurs REST.
2. **Construire une interface de pilotage** des tests de contrat, équivalent de l'interface web du TP front (bouton « Exécuter », historique, rapport) : **TUI** (terminal interactif, par exemple Rich/Textual en Python ou Ink en Node) ou **GUI** (petite appli web), au choix. L'important est de piloter les runs et d'en consulter le rapport sans repasser par la ligne de commande à chaque fois.
3. **Décrire** en langage naturel le comportement attendu d'un endpoint → l'agent **génère un test de contrat** depuis le schéma OpenAPI, exécutable depuis l'interface. Alternative consumer-driven : Pact, l'agent génère le pacte consumer / provider.

```bash
pip install schemathesis
st run openapi.yaml --base-url http://localhost:8080 --checks all
```

4. **Exécuter** depuis l'interface et observer : requêtes / réponses tracées dans le rapport Schemathesis, écarts de schéma mis en évidence.
5. **Historique des runs** : liste des derniers passages avec leur statut, affichée dans l'interface.
6. **Consulter le rapport** :
   - **succès** → statistiques (nombre de cas générés, endpoints couverts) ;
   - **échec** → outil d'analyse de cause assisté par IA :
     - **vraie régression de contrat** (un champ disparaît du schéma sans bump de version) → remonter le bug ;
     - **évolution intentionnelle de l'API** (nouvel endpoint versionné, par exemple `/v2/...`) → l'IA **met à jour le test de contrat automatiquement**.
7. **Pas de CI ici**, mais ce run pourrait être **planifié en CI** (à chaque PR, la nuit), comme pour la variante front.

Pendant la génération, vérifiez commit par commit via `doc-tracker` , à l'identique de la variante front :

1. **Doc à jour** : les endpoints testés sont-ils couverts dans `SPEC.md` (§5 Functional Behavior) ?
2. **Logs lisibles** : le serveur émet-il des logs structurés sur les actions clés (requête reçue, validation de schéma échouée) ?
3. **Traçabilité IA** : chaque commit indique-t-il la part de code générée automatiquement ?

> Même structure, même durée, même fil rouge `doc-tracker` : le **terrain d'exemple** change, pas l'apprentissage.

SUPPORT D'EXERCICE

Lab 6 · Rétro-documentation

Partir d'un projet legacy sans documentation, en extraire les spécifications, les injecter dans un outil SDD, puis recréer l'application. Identique pour GitHub Copilot et Claude Code.

1. **Projet conseillé** : `spring-petclinic-angularjs` (AngularJS 1 + Spring Boot 2) : domaine déjà connu, **front legacy** (AngularJS en fin de vie). Tout petit projet legacy maîtrisé convient (PHP, jQuery…). Prévoyez ses prérequis (JDK + Maven + Node).
2. **Rétro-spécifier avec Reversa** : l'IA s'appuie sur une **vraie méthodologie** (14 sous-agents, pipeline en phases).

```bash
npx reversa install     # détecte la stack, prépare la structure
# puis, dans l'agent : /reversa  → specs écrites dans _reversa_sdd/
```

3. **Relire** les specs générées, les **passer dans Spec Kit**, puis **régénérer** le front en **stack moderne** (Angular, React).
4. **Comparer** : l'app régénérée respecte-t-elle l'**intention** du legacy ?

> Reversa **ne touche pas** au code existant (il écrit dans `.reversa/` et `_reversa_sdd/`) : il **documente** avant de régénérer.

SUPPORT D'EXERCICE

Lab 7 · Poste terminal-first (facultatif)

Lab facultatif : mettre en place et utiliser un espace de travail terminal-first (agent, navigateur de fichiers, client git) pour développer une feature de la PetClinic.

1. Installer les outils

- tmux : https://github.com/tmux/tmux
- yazi : https://github.com/sxyazi/yazi
- lazygit : https://github.com/jesseduffield/lazygit

```bash
brew install tmux yazi lazygit
brew install ffmpegthumbnailer fd ripgrep fzf zoxide  # prévisualisation yazi
```

2. Prendre ses repères dans le CLI

GitHub Copilot CLI

Tapez `/` pour lister les commandes disponibles, puis :

```text
/model      # choisir le modèle : arbitrage coût / puissance
/context    # état de la context window
/diff       # ce que l'agent a modifié depuis le début de la session
```

Claude Code

Configurez Claude Code au terminal : une ligne d'état qui affiche le contexte utile, puis les réglages.

```text
/statusline Display current directory and/or username@hostname, the Active Claude model, the
/config
```

3. Monter le poste et développer

1. **Créez une session tmux** avec 3 panneaux (agent / yazi / lazygit) :
   - panneau principal : lancez `copilot` ou `claude` sur la PetClinic ;
   - panneau yazi : naviguez dans `src/main/java/` ;
   - panneau lazygit : surveillez les changements git en temps réel.
2. **Demandez à l'agent** d'implémenter une petite feature (par exemple : ajouter un champ `phone` à `Owner`), en observant les trois panneaux.
3. **Relisez dans lazygit** avant de committer.

> Le workflow complet : agent génère → yazi confirme → lazygit valide → commit.

Piège classique : tmux qui ne se découpe pas parce que le préfixe a été personnalisé. Ayez un `tmux.conf` minimal de référence sous la main. Équipé d'un autre CLI (`gemini` …) ? Lancez simplement le vôtre dans le panneau agent : le reste du poste est identique.

SUPPORT D'EXERCICE

Lab 8 · Développement parallèle

Développer deux features indépendantes de la PetClinic en parallèle, avec deux agents sur deux git worktrees, puis les réconcilier. Identique pour GitHub Copilot et Claude Code : seul le CLI lancé dans chaque terminal change.

1. **Créez deux worktrees** depuis la branche `main` de la PetClinic :

```bash
git worktree add ../petclinic-feat-a -b feat/clinic-list
git worktree add ../petclinic-feat-b -b feat/owner-search
```

2. **Recopiez les fichiers non versionnés** dans chaque worktree (`.env` , secrets, configuration locale) : ils ne suivent pas, et sans eux l'application ne démarre pas. L'agent partirait alors en boucle de debug sur un faux problème.
3. **Feature A, page « All Clinics »** : ouvrez un terminal dans `../petclinic-feat-a` , lancez `copilot` ou `claude` , demandez d'implémenter la page selon la user story du lab 2.2.
4. **Feature B, recherche d'owners** : ouvrez un terminal dans `../petclinic-feat-b` , lancez un second agent, demandez le filtre de recherche.
5. **Laissez les deux agents tourner** (surveillez lazygit dans chaque worktree).
6. **Réconciliez** :
   - créez des PR depuis chaque branche ;
   - mergez A → résolvez les éventuels conflits avec B → mergez B ;
   - nettoyez : `git worktree remove` + `git branch -d` .

> Comparez le **temps total** avec une approche séquentielle estimée. Identifiez les fichiers qui auraient conflicté.

> **Variante équipes backend** : deux features vraiment indépendantes côté API fonctionnent tout aussi bien, par exemple **Feature A'**, un nouvel endpoint `GET /clinics/{id}/stats` (agrégation simple), et **Feature B'**, un job de traitement asynchrone `vet-availability-sync` . Le point pédagogique (isolation de contexte, conflits de merge, réconciliation) ne dépend pas de la couche touchée.

> Côté Copilot, le CLI ne gère pas les worktrees : on passe par git, le geste le plus portable. L'**app Copilot** (desktop) place chaque session parallèle dans son propre worktree isolé, création et nettoyage compris.

SUPPORT D'EXERCICE

Lab 9 · Skill d'équipe

Produire un skill de qualité publication, le tester en session fraîche, puis le partager dans un registre simulé.

1. **Choisissez une expertise** encodable en skill : `reviewing-prs`, `generating-specs`, `analysing-e2e-failures`, `documenting-api-endpoints`.
2. **Écrivez le skill** en appliquant les bonnes pratiques :
   - frontmatter YAML : `name` (gérondif), `description` (troisième personne, **ce que ça fait** et **quand l'utiliser**) ;
   - corps : 500 lignes au plus, le contexte que l'agent ne peut pas déduire seul.
3. **Copilot : explorez ce qui existe déjà** avant de réinventer :

```bash
gh skill search "code review"
gh skill install github/awesome-copilot                          # explorer le dépôt
gh skill install github/awesome-copilot documentation-writer  # un skill précis
```

Un skill installé, c'est **du prompt exécuté chez vous** (et parfois des scripts) : relisez-le comme n'importe quelle dépendance tierce.

4. **Testez en session fraîche** :
   - ouvrez une **nouvelle session** (`copilot`, ou une nouvelle instance Claude, « Claude B »), sans contexte du skill ;
   - demandez la tâche couverte par le skill, **sans le nommer** ;
   - résultat attendu : le skill se déclenche automatiquement et correctement.
5. **Proposez-le en PR** sur le dépôt partagé du groupe :
   - créez `.github/skills/<nom-du-skill>/SKILL.md` (Copilot) ou `.claude/skills/<nom-du-skill>/SKILL.md` (Claude Code) dans un dépôt de démo ;
   - Copilot : ajoutez un `CODEOWNERS` et un tag de version (`v1.0.0`), c'est ce qui rend l'installation **reproductible** via `gh skill install <repo> <skill>@v1.0.0` ;
   - rédigez une description claire dans le message de PR.
6. **Revue croisée** : chaque participant relit le skill d'un autre. La `description` est-elle suffisamment précise ? Le corps, trop bavard ?

> Un skill est prêt quand une **session fraîche** l'utilise **sans coaching**, pas avant.

> Groupe mixte en outillage : faites tester le skill par un participant équipé d'un **autre agent** (Claude Code, Cursor…). Le format `SKILL.md` étant commun et Copilot lisant aussi `.claude/skills/`, ça fonctionne, et ça démontre la portabilité mieux qu'un slide.